

DEVELOPER REFERENCE GUIDE

Claude Token Optimization

Practical strategies to reduce token consumption, control context window, and maximize Claude's effectiveness in VS Code workflows.

Cisco AI & Networking Team · 2026

01 — Why Token Optimization Matters

Claude is stateless — every message re-transmits the entire conversation history. As sessions grow, token costs compound silently. Understanding this is the single most important step toward efficient AI-assisted development.

The Core Constraint

Claude's context window holds everything: messages, file reads, tool outputs.
Performance degrades as context fills — Claude 'forgets' earlier instructions.
Every file read, every command output, every response consumes tokens.
On API plans: token cost = money. On subscription: token cost = quota.

Scenario	Approx. Tokens	Impact
5,000-token CLAUDE.md	5,000 / session start	Constant baseline every session
Long debug session (no /compact)	20,000–50,000+	Context degrades, mistakes increase
Reading full codebase	10,000–30,000	Often only 2–3 files actually needed
Using /compact mid-session	~200 tokens saved context	Restores clarity, cuts cost
Targeted .md file vs chat history	300 vs 15,000+	50–70x more efficient

Key Principle

Token optimization is not about being cheap — it's about being smart. Lean context = better responses + lower cost + longer productive sessions. The strategies below work together; even applying 2–3 will produce noticeable results.

02 — The .md File Strategy (Highest Impact)

Instead of relying on chat history across sessions, create structured Markdown files that Claude reads on demand. This replaces thousands of tokens of conversation with a few hundred tokens of curated context.

Recommended File Structure

File	Purpose	When to Read
CLAUDE.md	Project conventions, stack, forbidden actions	Every session — auto-loaded
feature_implementation_plan.md	HLD, LLD, decisions, open questions	On-demand per feature
session_notes.md	Last session progress + next steps	Start of continuation session
decisions_log.md	Architecture choices + rationale (why)	When revisiting design
api_contracts.md	Endpoint schemas, response structures	During API integration work

Session Prompt Pattern

```
# Starting a session:
Read feature_implementation_plan.md and continue from where we left off.

# Ending a session:
Update feature_implementation_plan.md with today's progress.
Add a '3-bullet Next Session Start Here' section at the top.

# Before closing:
/cost
```

What to include in feature_implementation_plan.md

- Problem statement + scope boundaries (what is OUT of scope)
- High-Level Design: components, data flow, interfaces
- Low-Level Design: function signatures, DB schema, edge cases
- Key decisions made — AND why (rationale prevents re-debate)
- Current Status: Done / In Progress / Pending (updated each session)
- Open Questions: unresolved items for next session

Keep .md Files Lean — Prune Regularly

Files that grow beyond ~2,000 tokens defeat their own purpose.

Periodically: 'Summarize and prune this file — archive completed sections.'

Keep only forward-looking context. Past completions add no value.

03 — CLAUDE.md — Your Team's Force Multiplier

CLAUDE.md is auto-loaded at every session start. Every token here is paid on every turn. Make it earn its weight — include only what Claude needs to know for every task, not everything about your project.

Recommended CLAUDE.md Template

```
## Stack
Python 3.11, FastAPI, PostgreSQL, Redis, SONiC (Debian base)

## Conventions
- snake_case for all variables and functions
- All DB queries via repository layer – never direct ORM in routes
- Use existing utils/ before writing new helpers

## File Access Rules
- Do NOT read /tests unless explicitly asked
- Do NOT read /docs or /migrations unless relevant to the task
- List files you intend to read before reading them

## What NOT to Do
- Never modify schema.py directly – use Alembic migrations
- Never rewrite a function unless asked – patch only what's broken
- Never add explanatory comments unless asked
```

The 'What NOT to Do' Section

This is the highest ROI section of CLAUDE.md. Explicit prohibitions prevent Claude from doing expensive, unnecessary work — reading test suites, rewriting working code, or scanning migrations during a feature task.

04 — Slash Commands Reference

Command	What It Does	When to Use
<code>/compact</code>	Summarizes history, compresses context	Mid-session when context grows large
<code>/clear</code>	Full reset — clears entire context	Switching tasks, starting fresh
<code>/cost</code>	Shows token usage for current session	End of session — track consumption
<code>@filename</code>	Reference a specific file directly	Avoid broad directory exploration
<code>/model sonnet</code>	Switch to Sonnet mid-session	Implementation phase after Opus planning
<code>/model haiku</code>	Switch to Haiku mid-session	Boilerplate, tests, docs generation

05 — Prompt Engineering for Token Efficiency

Be Surgical, Not Conversational

```
# INEFFICIENT — triggers broad codebase scan:
X 'Look at my codebase and figure out why auth is broken.'

# EFFICIENT — scoped, targeted, explicit:
✓ 'Read auth/jwt_handler.py only.'
✓ 'The verify_token() function returns None for valid tokens.'
✓ 'Fix only that function. Return only the modified function.'
```

Scope Output Explicitly

```
✓ 'Return only the modified function, not the entire file.'
✓ 'No explanatory comments in the code.'
✓ 'Skip the summary at the end.'
✓ 'Respond in diff format only.'
✓ 'List the files you need to read. Wait for my confirmation.'
```

Use Git as a Token-Efficient Context Anchor

```
# Instead of opening files, use git state:
✓ 'Read the last 3 commit messages and current git diff.'
✓ 'That is all the context you need for this fix.'

# git diff + git log are tiny vs. reading full source files
```

06 — Model Selection Strategy

Don't run every task through Opus. Choose the right model for the phase of work. Opus is ~5x more expensive per token than Haiku at current pricing.

Model	Best For	Avoid For
Opus	Architecture planning, complex reasoning, HLD/LLD	Implementation, boilerplate, tests
Sonnet	Most coding tasks — best cost/quality balance	Very simple edits (use Haiku)
Haiku	Boilerplate, repetitive edits, test gen, docs	Novel design, debugging, analysis

Optimal Phase-Based Model Switching

1. /model opus → Planning, architecture decisions, HLD review
2. /model sonnet → Feature implementation, bug fixes
3. /model haiku → Test generation, docstrings, repetitive scaffolding

07 — Advanced Techniques

Use Subagents for Codebase Exploration

Instead of letting Claude scan files freely during your main session (burning main-session tokens), delegate exploration to a subagent. The subagent consumes its own context; your main session stays clean.

```
# Delegate exploration – don't pollute main context:
'Use a subagent to find all Redis cache invalidation points.
Summarize the pattern. Do not make any changes.'

# Main session receives a compact summary, not raw file reads
```

Declare Read Plan Before Acting

One of the most effective single habits. Forces Claude to be intentional about file access instead of doing a speculative broad scan.

```
# Add this to every task prompt:
'Before you start, list every file you intend to read.
Wait for my confirmation before reading any of them.'

# Saves: prevents 5-10 unnecessary file reads per session
```

Session Context Chunking

For large features, divide work into context-sized chunks with natural breakpoints rather than pushing one session to exhaustion. Each chunk ends with an update to the feature .md file.

```
Session 1: Design + HLD → update feature_plan.md
Session 2: Core logic → update feature_plan.md
Session 3: Edge cases → update feature_plan.md
Session 4: Tests + docs → close out feature_plan.md
```

Tool Output Size Control

Tool outputs (test runs, linter output, command results) flood context fast.

Cap tool output: 'Run tests, return only failing test names and error lines.'

For shell commands: pipe through head -20 or grep for relevant lines only.

08 — Anti-Patterns to Avoid

Anti-Pattern	Why It's Costly	Fix
'Look at my whole project and...'	Triggers broad file scan — 10,000+ tokens	Name specific files explicitly
Long debug sessions without /compact	Context balloons; quality degrades silently	Run /compact every ~30 mins or when slow
Keeping stale chat open across days	Old context is dead weight — Claude carries it	/clear at start of each new day/task
Asking Claude to explain its own code	Re-reads files + generates verbose explanation	Read code yourself; ask specific questions only
Reading test files during feature work	Tests rarely needed mid-implementation	Exclude /tests in CLAUDE.md by default
Verbose CLAUDE.md (5,000+ tokens)	Paid on every single turn of every session	Keep under 500 tokens; prune quarterly
Not using /cost at session end	No visibility into consumption patterns	Run /cost as last command every session

09 — End-of-Session Ritual

This 3-step routine takes 60 seconds and ensures the next session starts in ~300 tokens instead of re-reading 20,000 tokens of history.

```
# Step 1 – Persist progress to .md file:
'Update feature_implementation_plan.md with today's progress.
Mark completed items. List blockers.'

# Step 2 – Set the next session anchor:
'Add a 3-bullet Next Session Start Here section at the top.'

# Step 3 – Track consumption:
/cost
```

10 — Quick Reference Summary

Impact Matrix

Strategy	Token Saving	Effort	Start Here?
Use .md files instead of chat history	★★★★★	Low	✓ Yes
/clear between tasks	★★★★★	Minimal	✓ Yes
/compact mid-session	★★★★■	Minimal	✓ Yes
Lean CLAUDE.md (<500 tokens)	★★★★■	Low	✓ Yes
Right model per task phase	★★★★■	Low	✓ Yes
Declare read plan before acting	★★★☆☆	Low	Soon
Subagents for exploration	★★★☆☆	Medium	Soon
Session context chunking	★★★☆☆	Low	Soon
Tool output size control	★★☆☆■	Low	Later

The Golden Rules

5 Rules to Follow Starting Today

1. Create a feature_implementation_plan.md for every non-trivial feature.
2. Run /clear when switching tasks. Never carry stale context.
3. Run /compact when sessions feel slow or responses lose accuracy.
4. Keep CLAUDE.md under 500 tokens. Include 'What NOT to Do'.
5. Run /cost at session end. What gets measured gets optimized.

Expected Outcome

Applying strategies 1–5 above: 50–70% token reduction on complex features.
Session quality improves — Claude stays accurate longer per session.
Team develops a shared context discipline via .md files (like HLD/LLD for AI).